

PROJEKTAUFGABE

Event-Driven Cafe Order System

Delegates, Events und EventHandler in C#

Lernziel

Nicht nur die Syntax anwenden, sondern den architektonischen Sinn von Delegates und Events verstehen: Verhalten übergeben, Änderungen melden und Klassen lose miteinander koppeln.

Projekttyp	C# Console Application
Schwerpunkt	Delegate - Event - EventHandler<TEventArgs> - Lambda
Umfang	Kleines Lernprojekt mit mehreren Ausbaustufen

1. Ziel des Projekts

Das Projekt soll zeigen, warum Delegates und Events in einer echten Architektur sinnvoll sind. Im Mittelpunkt steht nicht eine möglichst große Anwendung, sondern eine klare Trennung der Verantwortlichkeiten.

Nach Abschluss des Projekts sollst du erklären können:

- warum Delegates nützlich sind und Verhalten als Wert darstellen können,
- wann ein Event sinnvoll ist,
- was Publisher und Subscriber sind,
- warum Events eine lose Kopplung zwischen Klassen ermöglichen,
- wie EventHandler<EventArgs> als .NET-Standard verwendet wird,
- worin sich Delegate und Event architektonisch unterscheiden.

Zentrale Idee

Ein Objekt soll mitteilen können, DASS etwas passiert ist, ohne wissen zu müssen, WER darauf reagiert und WAS danach ausgeführt wird.

2. Projektszenario

Entwickle ein einfaches Konsolenprogramm zur Bearbeitung von Bestellungen in einem Cafe. Eine Bestellung durchläuft mehrere Zustände.

```
New
|
v
Confirmed
|
v
Cooking
|
v
Ready
|
v
Completed
```

Zusätzlich kann eine Bestellung in passenden Zuständen storniert werden:

```
New / Confirmed / Cooking -> Cancelled
```

Im System existieren mehrere unabhängige Komponenten, die auf Statusänderungen reagieren können:

```
Order
|
+-- KitchenDisplay
+-- WaiterDisplay
+-- CustomerNotifier
+-- OrderLogger
+-- Statistics
```

Die Klasse Order soll diese Komponenten später nicht direkt kennen. Sie soll nur mitteilen, dass sich ihr Zustand geändert hat.

3. Technische Rahmenbedingungen

Das Projekt wird als C# Console Application umgesetzt. Der Fokus liegt ausschließlich auf Sprache, OOP und Architektur.

Nicht verwenden:

- ASP.NET
- Datenbank / Entity Framework

- Dependency-Injection-Framework
- grafische Benutzeroberfläche

Verwenden bzw. üben:

- Klassen und Objekte
- Delegates
- Events
- EventHandler<EventArgs>
- Lambdas
- saubere Verantwortlichkeiten zwischen Klassen

4. Grundmodell: Klasse Order

Erstelle eine Klasse Order mit mindestens folgenden Eigenschaften:

```
Id
CustomerName
TotalPrice
Status
```

Für den Status soll ein Enum verwendet werden, zum Beispiel:

```
enum OrderStatus
{
    New,
    Confirmed,
    Cooking,
    Ready,
    Completed,
    Cancelled
}
```

Die Klasse Order soll Methoden zur Änderung ihres Zustands besitzen, zum Beispiel:

```
Confirm()
StartCooking()
MarkReady()
Complete()
Cancel()
```

Order muss selbst prüfen, ob ein Zustandswechsel gültig ist. Beispiel: Cooking -> Ready ist erlaubt, Completed -> Cooking dagegen nicht.

5. Phase 1 - Bewusst ohne Events

Implementiere zuerst bewusst eine einfache, stark gekoppelte Variante. Nach einer Statusänderung werden alle notwendigen Reaktionen manuell ausgelöst.

```
Bestellung ändern
|
+--> Logger informieren
+--> KitchenDisplay aktualisieren
+--> WaiterDisplay aktualisieren
+--> Kunden informieren
+--> Statistik aktualisieren
```

Diese erste Version ist absichtlich nicht die endgültige Architektur. Sie dient dazu, das Problem zu erkennen, das Events später lösen sollen.

Analysefragen

1. Welche Klasse muss alle anderen Komponenten kennen?
2. Was muss geändert werden, wenn ein weiterer Empfänger hinzukommt?
3. Wie stark sind die Klassen voneinander abhängig?
4. Welche Klasse wird immer größer, wenn neue Reaktionen hinzukommen?

6. Phase 2 - Eigener Delegate

Erstelle zunächst einen eigenen Delegate. Er wird in dieser Phase noch nicht als Event verwendet.

```
public delegate void OrderAction(Order order);
```

Erstelle eine Methode, die eine Bestellung und ein Verhalten als Parameter erhält. Die konkrete Signatur darfst du selbst festlegen. Konzeptuell:

```
ExecuteForOrder(Order order, OrderAction action)
```

Übergib unterschiedliche Methoden als Verhalten, zum Beispiel:

- PrintOrder
- SendToKitchen
- CalculateSomething

Verwende danach mindestens einmal eine Lambda Expression anstelle einer benannten Methode.

Lernziel dieser Phase

Ein Delegate ermöglicht es, Verhalten bzw. eine Methode als Wert zu speichern, zu kombinieren oder an eine andere Methode zu übergeben.

7. Phase 3 - Event: StatusChanged

Erweitere Order um ein Ereignis StatusChanged. Ab diesem Zeitpunkt darf Order nicht mehr direkt wissen, welche Komponenten auf eine Statusänderung reagieren.

```
Order --Event--
      |-- KitchenDisplay
      |
      |-- WaiterDisplay
      |
      |-- OrderLogger
      |-- CustomerNotifier
      |-- Statistics
```

Wenn beispielsweise order.StartCooking() ausgeführt wird, ändert Order zuerst seinen eigenen Zustand und meldet anschließend das Ereignis StatusChanged.

Wichtig: Order entscheidet nicht, wer auf dieses Ereignis reagiert.

8. Phase 4 - EventHandler<TEventArgs>

Stelle das Event anschließend auf die übliche .NET-Konvention mit EventHandler<TEventArgs> um.

```
EventHandler<OrderStatusChangedEventArgs>
```

Erstelle dafür die Klasse OrderStatusChangedEventArgs. Sie soll mindestens enthalten:

- OldStatus
- NewStatus
- ChangedAt

Optional kann zusätzlich die OrderId enthalten sein. Beispiel für die transportierten Informationen:

```
Order #125
Confirmed -> Cooking
10:42
```

Gedanke hinter EventArgs

Das Event meldet nicht nur, DASS etwas passiert ist, sondern kann den Subscribern die notwendigen Informationen über das Ereignis mitgeben.

9. Subscriber

Implementiere mindestens vier voneinander unabhängige Subscriber. Alle hören auf dasselbe Event, dürfen aber unterschiedlich reagieren.

9.1 KitchenDisplay

Reagiert auf: Confirmed, Cooking, Cancelled.

```
KITCHEN: Order #125 started cooking.
```

9.2 WaiterDisplay

Reagiert auf: Ready, Completed.

```
WAITER: Order #125 is ready.
```

9.3 OrderLogger

Reagiert auf: jede Statusänderung.

```
[10:42] Order #125: Confirmed -> Cooking
```

9.4 CustomerNotifier

Reagiert auf: Confirmed, Ready, Cancelled.

```
CUSTOMER: Your order is ready.
```

Optional: Implementiere zusätzlich eine Klasse Statistics, die beispielsweise abgeschlossene und stornierte Bestellungen zählt.

Wichtige Beobachtung

Ein Event kann mehrere Subscriber haben. Jeder Subscriber entscheidet selbst, ob und wie er auf die erhaltene Information reagiert.

10. Dynamisches An- und Abmelden

Zeige während der Programmlaufzeit, dass Subscriber registriert und wieder entfernt werden können.

```
+ = // Subscriber registrieren
- = // Subscriber entfernen
```

Beispiel: CustomerNotifier wird registriert, erhält einige Meldungen und wird anschließend wieder entfernt. Die Bestellung muss danach normal weiterarbeiten - nur ohne weitere Kundenbenachrichtigungen.

Beobachte dabei: Order weiß nicht, wie viele Subscriber existieren, welche Subscriber existieren oder ob ein bestimmter Subscriber aktuell registriert ist.

11. Phase 5 - Ein zweites Event

Erweitere das System um ein zusätzliches Event, zum Beispiel OrderCancelled. Vergleiche anschließend zwei mögliche Architekturen:

```
Variante A:
    StatusChanged

Variante B:
    StatusChanged
    OrderReady
    OrderCancelled
```

Entscheide selbst, welche Variante für dein Projekt sinnvoller ist, und begründe deine Entscheidung.

Leitfrage: Wann ist ein spezialisiertes Event hilfreich - und wann führt eine große Anzahl von Events nur zu unnötiger Komplexität?

12. Phase 6 - Delegates außerhalb von Events

Verwende Delegates zusätzlich für eine Aufgabe, die nichts mit Events zu tun hat: Filterung von Bestellungen.

Nutze dafür beispielsweise:

```
Predicate<Order>
oder
Func<Order, bool>
```

Mögliche Filter:

- Bestellungen über 50 EUR
- fertige Bestellungen
- stornierte Bestellungen
- Bestellungen eines bestimmten Kunden

Beispiel für ein Kriterium:

```
o => o.TotalPrice > 50
o => o.Status == OrderStatus.Ready
```

Lernziel

Delegates sind ein allgemeines Sprachmittel zur Übergabe von Verhalten. Ein Delegate muss nicht automatisch etwas mit Events zu tun haben.

13. Verbindliche Architekturregeln

Die Klasse Order darf in der finalen Event-Version keine direkten Referenzen auf folgende Komponenten besitzen:

- KitchenDisplay
- WaiterDisplay
- CustomerNotifier
- OrderLogger
- Statistics

Folgendes Prinzip ist innerhalb von Order ausdrücklich zu vermeiden:

```
logger.Log(...);
notifier.Send(...);
kitchen.Update(...);
```

Die Verantwortlichkeiten sollen klar getrennt sein:

Order	eigenen Zustand verwalten und Änderungen bekannt geben
Event	Information über eine eingetretene Änderung transportieren
Subscriber	auf die Änderung reagieren

14. Mögliche Projektstruktur

```

CafeOrderEvents
|
+-- Models
|   +-- Order.cs
|   +-- OrderStatus.cs
|
+-- Events
|   +-- OrderStatusChangedEventArgs.cs
|
+-- Subscribers
|   +-- KitchenDisplay.cs
|   +-- WaiterDisplay.cs
|   +-- CustomerNotifier.cs
|   +-- OrderLogger.cs
|   +-- Statistics.cs
|
+-- Program.cs

```

Die konkrete Struktur darf verändert werden, wenn die Entscheidung logisch begründet werden kann.

15. Pflicht-Test-Szenario

Das Programm soll mindestens folgenden Ablauf demonstrieren:

```

Order #125 wird erstellt.

New -> Confirmed -> Cooking -> Ready -> Completed

```

Währenddessen sollen verschiedene Subscriber reagieren, zum Beispiel:

```

[LOG] Order #125: New -> Confirmed
[KITCHEN] New order received.

[LOG] Order #125: Confirmed -> Cooking
[KITCHEN] Cooking started.

[LOG] Order #125: Cooking -> Ready
[WAITER] Order #125 is ready.
[CUSTOMER] Your order is ready.

[LOG] Order #125: Ready -> Completed
[STATISTICS] Order completed.

```

Zusätzlich muss mindestens ein ungültiger Zustandswechsel getestet und abgelehnt werden, zum Beispiel Completed -> Cooking.

16. Reflexionsfragen

Nach Abschluss des Projekts solltest du folgende Fragen ohne auswendig gelernte Definitionen beantworten können.

Delegates

5. Was ist ein Delegate?
6. Warum kann man sagen, dass ein Delegate "Verhalten speichert"?
7. Warum kann ein Delegate als Methodenparameter verwendet werden?
8. Was ist ein Multicast Delegate?

Events

9. Was ist der Unterschied zwischen einem Delegate und einem Event?
10. Was bedeuten Publisher und Subscriber?
11. Warum reduziert ein Event die Kopplung zwischen Klassen?
12. Warum verwendet man bei Events += und -=?

EventHandler

13. Warum existiert EventHandler<EventArgs>?
14. Welche Bedeutung hat sender?
15. Welche Informationen gehören in EventArgs?

Architektur

16. Warum sollte Order keinen CustomerNotifier kennen?
17. Was passiert mit Order, wenn später ein neuer Subscriber hinzukommt?
18. Welche Vorteile hat diese Architektur gegenüber direkten Methodenaufrufen?

17. Abschluss - Was soll nach dem Projekt klar sein?

Delegate

Ich möchte festlegen oder übergeben, WAS ausgeführt werden soll.

Event

Ein Objekt meldet, DASS etwas passiert ist.

EventHandler<EventArgs>

Standardisierte .NET-Struktur: WER hat das Ereignis ausgelöst und WAS ist passiert?

Die wichtigste Aussage des Projekts

Der Publisher kennt seine Subscriber nicht.

Eine Bestellung weiß nur: "Mein Status hat sich geändert." Sie weiß nicht, wer zuhört, warum jemand zuhört oder was daraufhin ausgeführt wird. Genau darin liegt der architektonische Nutzen von Events.